

SC Assignment 2

Liam Erasmus

2026-03-03

Table of contents

Statistical Computing Assignment 2	3
1 Parallel Computing	4
1.1 Parallel Computing Prac	4
1.1.1 Question 1	4
1.1.2 Question 2	7
1.1.3 Question 3	8
1.1.4 Question 4	9
1.1.5 Question 5	9

Statistical Computing Assignment 2

Parallel Computing Practical

1 Parallel Computing

1.1 Parallel Computing Prac

1.1.1 Question 1

```
library(foreach)
library(kableExtra)
# Repeat 100 times: generate a sample from Exp(1) and compute mean and variance
one <- as.data.frame( foreach(i = 1:100, .combine = rbind) %do%{
  samp <- rexp(100, 1)
  c(
    x_bar <- mean(samp), # mean
    vari <- var(samp)) #variance
  #results = rbind(x_bar, vari)
})
)
kable(one,
  col.names = c("Sample Mean", "Sample Variance"))
```

	Sample Mean	Sample Variance
result.1	1.0242947	1.4021147
result.2	1.0008511	1.1218458
result.3	1.0923240	1.0279769
result.4	0.9546300	0.7132418
result.5	1.0888306	1.4767246
result.6	0.7897289	0.5631797
result.7	0.9822113	0.8524994
result.8	1.0001764	0.7650426
result.9	1.0914594	1.6323232
result.10	0.9650733	0.8434820
result.11	1.0391682	1.0308821
result.12	1.1040161	1.3295844
result.13	1.0690905	1.1571864

	Sample Mean	Sample Variance
result.14	0.8795467	0.7530651
result.15	0.7992830	0.5520129
result.16	1.1020837	0.9792511
result.17	0.9700011	0.9445461
result.18	1.1241766	1.2674073
result.19	0.9719095	1.0443217
result.20	0.9976525	1.0497840
result.21	0.8452708	0.5924539
result.22	0.9787509	0.7912277
result.23	1.0276468	1.6383848
result.24	1.1304874	1.6833558
result.25	1.0852426	1.0232031
result.26	0.9134181	1.0371452
result.27	1.1079240	1.2953870
result.28	1.1228784	1.4047668
result.29	1.0819128	0.8780869
result.30	1.1145837	1.3079122
result.31	1.2297040	1.3878704
result.32	0.7755863	0.4593755
result.33	0.8560176	0.7143574
result.34	1.0084491	1.2411827
result.35	1.0864492	1.5340530
result.36	1.0602961	1.1109602
result.37	1.1916553	0.9345824
result.38	0.8601058	0.8779851
result.39	1.0253868	1.0234971
result.40	0.9294457	0.9499390
result.41	0.9392847	0.7791009
result.42	0.7037850	0.4538308
result.43	0.9456492	0.8115876
result.44	1.0076414	0.8418271
result.45	1.0200323	1.0167683
result.46	1.2994977	1.6982834
result.47	0.9514566	0.8910819
result.48	0.9976911	0.6279103
result.49	1.3111236	1.5005588
result.50	0.9055792	0.6335447
result.51	1.0777229	0.6843847
result.52	1.1636898	1.2474290
result.53	0.9425866	0.6803895
result.54	1.0321518	0.7993825

	Sample Mean	Sample Variance
result.55	0.8629808	0.6139128
result.56	0.9830028	1.0424645
result.57	0.8109763	0.5517133
result.58	1.1103410	1.5922875
result.59	0.9044100	0.7622225
result.60	1.0184712	0.9083424
result.61	0.9685660	0.9696421
result.62	0.7938748	0.5673695
result.63	1.1760240	1.2815089
result.64	1.0082375	1.1859903
result.65	1.0741775	1.0030401
result.66	0.9161199	0.6509821
result.67	1.0672068	1.2079712
result.68	1.0706211	1.0645913
result.69	1.0288099	1.1714140
result.70	1.0701483	1.1709857
result.71	0.9650730	0.9541410
result.72	1.1801392	3.0914192
result.73	0.9325015	1.1518084
result.74	1.0410454	2.0558027
result.75	0.8370250	0.6580893
result.76	1.0466412	0.7197220
result.77	0.8586127	0.6744193
result.78	1.0480389	0.8111181
result.79	1.0118979	0.7864558
result.80	1.0112829	0.7238627
result.81	0.9941713	0.8449846
result.82	0.8902206	0.8909232
result.83	0.9523912	0.8423739
result.84	1.3062795	1.4237483
result.85	1.0482116	1.0655619
result.86	0.9868948	0.8015667
result.87	0.8746210	0.7446891
result.88	0.9129574	1.0824637
result.89	0.9622480	0.7646539
result.90	1.2174291	1.1669649
result.91	0.9260608	1.0930295
result.92	0.9659081	0.7944307
result.93	0.9413673	0.8380465
result.94	0.9977290	1.5154365
result.95	1.2173438	1.2616191

	Sample Mean	Sample Variance
result.96	0.9620882	1.0631415
result.97	1.0720925	0.9686941
result.98	1.0295351	1.2053053
result.99	1.0038039	1.1692028
result.100	1.0371071	0.8974095

1.1.2 Question 2

```
library(doParallel)
```

Loading required package: iterators

Loading required package: parallel

```
library(MASS)

x <- galaxies
n <- length(x)
B <- 200000

# Serial Bootstrap
t_serial <- system.time({
  med_serial <- replicate(B, median(sample(x, n, replace = TRUE)))
})["elapsed"]

t_serial
```

elapsed
2.507

```
# One Bootstrap per task (in parallel)

cl <- makeCluster(detectCores() - 1) # do not want my computer to crash
registerDoParallel(cl)

t_par_single <- system.time({
  med_par_single <- foreach(i = 1:B, .combine = c) %dopar% {
```

```

    median(sample(x, n, replace = TRUE))
  }
})["elapsed"]

stopCluster(cl)
t_par_single #MUCH SLOWER

```

```

elapsed
14.347

```

```

# 1000 Bootstraps per task
chunk_size <- 1000
nchunks <- 200

cl <- makeCluster(detectCores() - 1)
registerDoParallel(cl)

t_par_chunk <- system.time({
  med_par_chunk <- foreach(k = 1:nchunks, .combine = c) %dopar% {
    m <- if (k < nchunks) chunk_size else (B - (nchunks - 1) * chunk_size)
    replicate(m, median(sample(x, n, replace = TRUE)))
  }
})["elapsed"]

stopCluster(cl)

t_par_chunk # MUUUUCH FASTER

```

```

elapsed
0.474

```

1.1.3 Question 3

```

n <- 50 # sample size
mu_true <- 1 # true mean of Exp(1)
B <- 2000 # number of bootstrap samples
M <- 10000 # number of Monte Carlo simulations

cl3 <- makeCluster(detectCores() - 1) # again, do not crash

```



```

registerDoParallel(cl) # register cluster for parallel foreach

coverage_vec <- foreach(m = 1:M, .combine = c) %dopar% {

  x <- rexp(n, rate = 1)

  boot_means <- replicate(B, mean(sample(x, n, replace = TRUE)))

  ci <- quantile(boot_means, probs = c(0.025, 0.975), names = FALSE) # percentile bootstrap CI

  (ci[1] <= mu_true) && (mu_true <= ci[2]) # check whether the true mean lies inside the CI
}

stopCluster(cl3)

mean(coverage_vec) # estimated coverage probability

```

```
[1] 0.9272
```

1.1.4 Question 4

```

library(iterators)

set.seed(1234)

foreach(x = irnorm(3, count = 5), .combine = c) %do% {
  max(x) # max of each vector
}

```

```
[1] 1.0844412 0.5060559 -0.5466319 -0.4771927 0.9594941
```

1.1.5 Question 5

```

library(parallel)

R <- 50000
set.seed(1234)

```

```

# normal way
# Generate 3 samples of size 5 from N(0,1),
# take the maximum of each sample
t_rep <- system.time(replicate
                      (R, replicate
                       (3, max(rnorm(5))))))["elapsed"]

# with for each
# get the maximum of 3 samples
set.seed(1234)
t_for <- system.time(replicate(R,
  foreach(x = irnorm(3, count = 5), .combine = c) %do% max(x)
))["elapsed"]

# with parLapply
# distribute the 3 computations across cores
cl <- makeCluster(detectCores() - 1)
clusterSetRNGStream(cl, 1234)
t_par <- system.time(replicate(R,
  unlist(parLapply(cl, 1:3, function(i) max(rnorm(5))))
))["elapsed"]
stopCluster(cl)

c(replicate = t_rep, foreach = t_for, parLapply = t_par) # answers

```

```

replicate.elapsed    foreach.elapsed parLapply.elapsed
               0.369                24.314                15.973

```